

Le traitement Batch avec Hadoop HDFS , Map Reduce

1-Le traitement Batch avec Hadoop HDFS et Map Reduce

Objectifs du TP

Initiation au framework hadoop et au patron MapReduce, utilisation de docker pour lancer un cluster hadoop de 3 noeuds.

Hadoop et Docker

Pour déployer le framework Hadoop, nous allons utiliser des conteneurs [Docker](#). L'utilisation des conteneurs va garantir la consistance entre les environnements de développement et permettra de réduire considérablement la complexité de configuration des machines (dans le cas d'un accès natif) ainsi que la lourdeur d'exécution (si on opte pour l'utilisation d'une machine virtuelle).

Installation

```
sudo docker pull liliasfaxi/hadoop-cluster:latest
```

Ajouter ton utilisateur au groupe Docker (RECOMMANDÉE) :

```
sudo usermod -aG docker $USER
```

Redémarrer la session (IMPORTANT) :

```
sudo reboot
```

Créer les trois conteneurs à partir de l'image téléchargée. Pour cela:

Créer un réseau qui permettra de relier les trois conteneurs:

```
docker network create --driver=bridge hadoop
```

Vérification

```
docker network ls
```

Créer les 3 conteneurs Hadoop

Créer et lancer les trois conteneurs (les instructions -p permettent de faire un mapping entre les ports de la machine hôte et ceux du conteneur):

```
docker run -itd --net=hadoop -p 9870:9870 -p 8088:8088 -p 7077:7077 -p 16010:16010  
--name hadoop-master --hostname hadoop-master liliasfaxi/hadoop-cluster:latest
```

```
docker run -itd -p 8040:8042 --net=hadoop --name hadoop-worker1 --hostname  
hadoop-worker1 liliasfaxi/hadoop-cluster:latest
```

```
docker run -itd -p 8041:8042 --net=hadoop --name hadoop-worker2 --hostname hadoop-worker2 liliastaxi/hadoop-cluster:latest
```

verifier : `docker ps`

Entrer dans le conteneur master pour commencer à l'utiliser:

```
docker exec -it hadoop-master bash
```

Vous vous retrouverez dans le shell du namenode, et vous pourrez ainsi manipuler le cluster à votre guise. La première chose à faire, une fois dans le conteneur, est de lancer hadoop et yarn. Un script est fourni pour cela, appelé start-hadoop.sh. Lancer ce script.

```
./start-hadoop.sh
```

Interfaces web pour Hadoop

<http://localhost:9870>

<http://localhost:8088>

Créer un répertoire dans HDFS, appelé *input* :

```
hdfs dfs -mkdir -p input
```

Nous allons utiliser le fichier [purchases.txt](#) comme entrée pour le traitement MapReduce. Ce fichier se trouve déjà dans votre container, sous le répertoire principal (il suffit de taper `ls` et vous le verrez).

À partir du conteneur master, charger le fichier purchases dans le répertoire input (de HDFS) que vous avez créé:

```
hdfs dfs -put purchases.txt input
```

Pour afficher le contenu du répertoire *input*, la commande est:

```
hdfs dfs -ls input
```

Pour afficher les dernières lignes du fichier purchases:

```
hdfs dfs -tail input/purchases.txt
```

```
santoudllo@santoudllo-virtual-machine:~/Desktop/hadoop_avec_docker/bigdata$ docker exec -it hadoop-master bash
root@hadoop-master:~# hdfs dfs -ls input
Found 1 items
-rw-r--r-- 2 root supergroup 211312924 2026-01-02 23:13 input/purchases.txt
root@hadoop-master:~# hdfs dfs -tail input/purchases.txt
31      17:59  Norfolk Toys      164.34  MasterCard
2012-12-31  17:59  Chula Vista      Music   380.67  Visa
2012-12-31  17:59  Hialeah Toys     115.21  MasterCard
2012-12-31  17:59  Indianapolis     Men's Clothing 158.28  MasterCard
2012-12-31  17:59  Norfolk Garden   414.09  MasterCard
2012-12-31  17:59  Baltimore        DVDs    467.3   Visa
2012-12-31  17:59  Santa Ana        Video Games 144.73  Visa
2012-12-31  17:59  Gilbert Consumer Electronics 354.66  Discover
2012-12-31  17:59  Memphis Sporting Goods 124.79  Amex
2012-12-31  17:59  Chicago Men's Clothing 386.54  MasterCard
2012-12-31  17:59  Birmingham       CDs     118.04  Cash
2012-12-31  17:59  Las Vegas        Health and Beauty 420.46  Amex
2012-12-31  17:59  Wichita Toys     383.9   Cash
2012-12-31  17:59  Tucson Pet Supplies 268.39  MasterCard
2012-12-31  17:59  Glendale         Women's Clothing 68.05   Amex
2012-12-31  17:59  Albuquerque      Toys    345.7   MasterCard
2012-12-31  17:59  Rochester        DVDs    399.57  Amex
2012-12-31  17:59  Greensboro       Baby    277.27  Discover
2012-12-31  17:59  Arlington        Women's Clothing 134.95  MasterCard
2012-12-31  17:59  Corpus Christi   DVDs    441.61  Discover
root@hadoop-master:~#
```

Voici un tableau suivant les commandes les plus utilisées pour manipuler les fichiers dans HDFS:

Instruction	Fonctionnalité
<code>hdfs dfs -ls</code>	Afficher le contenu du répertoire racine
<code>hdfs dfs -put file.txt</code>	Upload un fichier dans hadoop (à partir du répertoire courant de votre disque local)
<code>hdfs dfs -get file.txt</code>	Download un fichier à partir de hadoop sur votre disque local
<code>hdfs dfs -tail file.txt</code>	Lire les dernières lignes du fichier
<code>hdfs dfs -cat file.txt</code>	Affiche tout le contenu du fichier
<code>hdfs dfs -mv file.txt newfile.txt</code>	Renommer (ou déplacer) le fichier
<code>hdfs dfs -rm newfile.txt</code>	Supprimer le fichier
<code>hdfs dfs -mkdir myinput</code>	Créer un répertoire

Vous pouvez également visualiser l'avancement et les résultats de vos Jobs (Map Reduce ou autre) en allant à l'adresse: <http://localhost:8088>.

Map Reduce

Présentation

Un Job Map-Reduce se compose principalement de deux types de programmes:

- **Mappers** : permettent d'extraire les données nécessaires sous forme de clef/valeur, pour pouvoir ensuite les trier selon la clef
- **Reducers** : prennent un ensemble de données triées selon leur clef, et effectuent le traitement nécessaire sur ces données (somme, moyenne, total...)

Exemple MapReduce avec **purchases.txt**

Objectif (simple à comprendre) : Compter le nombre d'achats par moyen de paiement (Visa, MasterCard, Cash, Amex, Discover...)

MAP transforme les données d'entrée

Il lit les données **ligne par ligne** et produit des **paires (clé, valeur)**.

clé : ce sur quoi on veut regrouper

valeur : une information associée à la clé

SHUFFLE regroupe toutes les paires qui ont la même clé.

REDUCE applique une fonction sur la liste de valeurs.

Dans notre exemple :
il **additionne** les valeurs.

Objectif du programme : **Compter le nombre d'achats par moyen de paiement**
à partir de :

input/purchases.txt (dans HDFS)

à coder : **PaymentCount.java**

créer un dossier de sorti exemple **mkdir pc**

Compilation et exécution : **javac -classpath \$(hadoop classpath) -d pc PaymentCount.java**

créer le fichier JAR :

Un JAR (Java ARchive) est : `jar -cvf paymentcount.jar -C pc/ .`
un fichier compressé

- qui contient tout le code Java compilé (.class)
- parfois aussi des fichiers de configuration

C'est l'équivalent d'un exécutable en Java.
soie :

```
• santoudllo@santoudllo-virtual-machine:~/Desktop/hadoop_avec_docker/scr$ jar -cvf paymentcount.jar -C pc/ .
added manifest
adding: PaymentCount$PaymentReducer.class(in = 1652) (out= 690)(deflated 58%)
adding: PaymentCount$PaymentMapper.class(in = 1817) (out= 726)(deflated 60%)
adding: PaymentCount.class(in = 1623) (out= 888)(deflated 45%)
• santoudllo@santoudllo-virtual-machine:~/Desktop/hadoop_avec_docker/scr$
```

copier le JAR dans le conteneur: `docker cp paymentcount.jar hadoop-master:/root/`

```
• santoudllo@santoudllo-virtual-machine:~/Desktop/hadoop_avec_docker/scr$ docker cp paymentcount.jar hadoop-master:/root/
Successfully copied 5.12kB to hadoop-master:/root/
```

lancer le job MapReduce (il faut être déjà dans le conteneur (root@hadoop-master)):
`hadoop jar paymentcount.jar PaymentCount input/purchases.txt output_payments`

Vérifier le résultat Après la fin du job :

`hdfs dfs -ls output_payments`

`hdfs dfs -cat output_payments/part-r-00000`

```
root@hadoop-master:~# hdfs dfs -ls output payments
hdfs dfs -cat output_payments/part-r-00000
Found 2 items
-rw-r--r--  2 root supergroup      0 2026-01-04 12:42 output_payments/ SUCCESS
-rw-r--r--  2 root supergroup    70 2026-01-04 12:42 output_payments/part-r-00000
Amex      826535
Cash      828770
Discover   827426
MasterCard 828524
Visa      827221
```

Stockage de Données dans une Base NOSQL avec HBase

Présentation

HBase est un système de gestion de bases de données distribué, non-relationnel et orienté colonnes, développé au-dessus du système de fichier HDFS. Il permet un accès aléatoire en écriture/lecture en temps réel à un très grand ensemble de données.

Modèle de données HBase

Le modèle de données de **Apache HBase** repose sur **six concepts fondamentaux** :

1. **Table**

Les données sont stockées dans des tables, identifiées par un nom (chaîne de caractères).

2. **Row (ligne)**

Chaque table contient des lignes, identifiées de manière unique par une **RowKey**. La RowKey n'est pas typée : elle est traitée comme un tableau d'octets (`byte[]`).

3. **Column Family (famille de colonnes)**

Les données d'une ligne sont regroupées par familles de colonnes.

- Toutes les lignes partagent les mêmes column families
- Elles sont définies **à la création de la table**
- Elles peuvent être vides ou non
- Le nom est une chaîne de caractères

4. **Column Qualifier (colonne)**

À l'intérieur d'une column family, les données sont accessibles via un **column qualifier**.

- Défini **lors de l'insertion**, pas à la création de la table
- Non typé, traité comme un tableau d'octets

5. **Cell (cellule)**

Une cellule est identifiée de façon unique par :

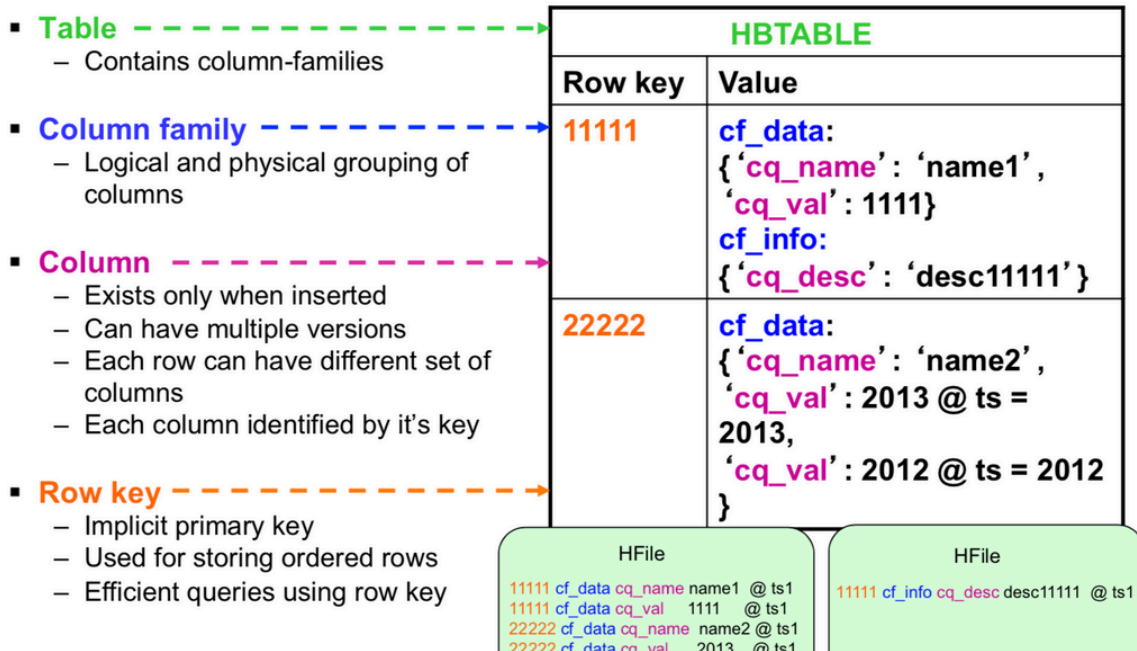
RowKey + Column Family + Column Qualifier

La valeur stockée dans une cellule est également non typée (`byte[]`).

6. **Version**

Les valeurs d'une cellule sont **versionnées**.

- Chaque version est identifiée par un **timestamp** (`long`)
- Le nombre de versions est configuré au niveau de la column family
- Par défaut, **3 versions** sont conservées



Architecture

Physiquement, HBase est composé de trois types de serveurs de type Master/Worker.

- **Region Servers:** permettent de fournir les données pour lectures et écritures. Pour accéder aux données, les clients communiquent avec les RegionServers directement.
- **HBase HMaster :** gère l'affectation des régions, les opérations de création et suppression de tables.
- **Zookeeper:** permet de maintenir le cluster en état.

Le DataNode de Hadoop permet de stocker les données que le Region Server gère. Toutes les données de HBase sont stockées dans des fichiers HDFS. Les RegionServers sont colocalisés avec les DataNodes.

Le NameNode permet de maintenir les métadonnées sur tous les blocs physiques qui forment les fichiers.

Installation

HBase est installé sur le même cluster que précédemment. Suivre les étapes décrites dans la partie *Installation* de **hadoop** pour télécharger l'image et exécuter les trois conteneurs. Si cela est déjà fait, il suffit de lancer vos machines grâce aux commandes suivantes:

docker start hadoop-master hadoop-worker1 hadoop-worker2

puis d'entrer dans le conteneur master: **docker exec -it hadoop-master bash**

Lancer ensuite les démons yarn et hdfs: **./start-hadoop.sh**

Lancer HBase en tapant : **start-hbase.sh**

Une fois c'est fait, en tapant **jps**, vous devriez avoir un résultat ressemblant au suivant:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

root@hadoop-master:~#jps
354 SecondaryNameNode
599 ResourceManager
168 NameNode
5400 HRegionServer
5913 Jps
5321 HMaster
5245 HQuorumPeer
root@hadoop-master:~#
```

Vous remarquerez que tous les démons Hadoop (NameNode, SecondaryNameNode et ResourceManager) ainsi que les démons HBase (HRegionServer, HMaster et HQuorumPeer (Zookeeper)) sont démarrés.

Première manipulation de HBase

Pour manipuler votre base de données avec son shell interactif, vous devez lancer le script suivant: **hbase shell**

Nous allons créer une base de données qui contient les données suivantes:

Row Key	customer		sales	
ROW_ID	name	city	product	amount
101	John White	Los Angeles, CA	Chairs	\$400.00
102	Jane Brown	Atlanta, GA	Lamps	\$200.00
103	Bill Green	Pittsburgh, PA	Desk	\$500.00
104	Jack Black	St. Louis, MO	Bed	\$1,600.00

Créer la table, ainsi que les familles de colonnes associées:

create 'sales_ledger','customer','sales'

Vérifier que la table est bien créée: **list**

```
hbase:004:0> list
TABLE
sales_ledger
1 row(s)
Took 0.0615 seconds
=> ["sales_ledger"]
hbase:005:0>
```


Insérer les différentes lignes:

```
put 'sales_ledger','101','customer:name','John White'  
put 'sales_ledger','101','customer:city','Los Angeles, CA'  
put 'sales_ledger','101','sales:product','Chairs'  
put 'sales_ledger','101','sales:amount','$400.00'
```

```
put 'sales_ledger','102','customer:name','Jane Brown'  
put 'sales_ledger','102','customer:city','Atlanta, GA'  
put 'sales_ledger','102','sales:product','Lamps'  
put 'sales_ledger','102','sales:amount','$200.00'
```

```
put 'sales_ledger','103','customer:name','Bill Green'  
put 'sales_ledger','103','customer:city','Pittsburgh, PA'  
put 'sales_ledger','103','sales:product','Desk'  
put 'sales_ledger','103','sales:amount','$500.00'
```

```
put 'sales_ledger','104','customer:name','Jack Black'  
put 'sales_ledger','104','customer:city','St. Louis, MO'  
put 'sales_ledger','104','sales:product','Bed'  
put 'sales_ledger','104','sales:amount','$1,600.00'
```

Visualiser le résultat de l'insertion : `scan 'sales_ledger'`

```
hbase:025:0> scan 'sales_ledger'  
ROW COLUMN+CELL  
101 column=customer:city, timestamp=2026-01-04T12:07:01.619, value=Los Angeles, CA  
101 column=customer:name, timestamp=2026-01-04T12:07:01.516, value=John White  
101 column=sales:amount, timestamp=2026-01-04T12:07:01.892, value=$400.00  
101 column=sales:product, timestamp=2026-01-04T12:07:01.778, value=Chairs  
102 column=customer:city, timestamp=2026-01-04T12:07:02.139, value=Atlanta, GA  
102 column=customer:name, timestamp=2026-01-04T12:07:02.054, value=Jane Brown  
102 column=sales:amount, timestamp=2026-01-04T12:07:02.318, value=$200.00  
102 column=sales:product, timestamp=2026-01-04T12:07:02.238, value=Lamps  
103 column=customer:city, timestamp=2026-01-04T12:07:02.536, value=Pittsburgh, PA  
103 column=customer:name, timestamp=2026-01-04T12:07:02.425, value=Bill Green  
103 column=sales:amount, timestamp=2026-01-04T12:07:02.765, value=$500.00  
103 column=sales:product, timestamp=2026-01-04T12:07:02.634, value=Desk  
104 column=customer:city, timestamp=2026-01-04T12:07:02.925, value=St. Louis, MO  
104 column=customer:name, timestamp=2026-01-04T12:07:02.872, value=Jack Black  
104 column=sales:amount, timestamp=2026-01-04T12:07:06.633, value=$1,600.00  
104 column=sales:product, timestamp=2026-01-04T12:07:02.994, value=Bed  
4 row(s)  
Took 0.1162 seconds
```